

CS201L: Artificial Intelligence Laboratory

Lab 10 – Regression with Multi-Input Variables

Indian Institute of Technology Dharwad

1 Problem Statement

You are given the dataset `EnergyEfficiency.csv`, which contains various building design parameters that affect the energy efficiency of residential buildings. The dataset is generated from simulations performed using Ecotect on different building configurations.

The objective of this task is to build regression models that learn the relationship between building structural features and the energy consumption of the building.

The goal is to predict the building's energy demand in terms of **Heating Load** using the available building design features.

2 Dataset Details

Input Variables:

- x_1 : Relative Compactness
- x_2 : Surface Area
- x_3 : Wall Area
- x_4 : Roof Area
- x_5 : Overall Height
- x_6 : Orientation
- x_7 : Glazing Area
- x_8 : Glazing Area Distribution
- x_9 : Cooling Load

Output Variables:

- y : Heating Load

Note: Different regression models will be built using:

- A single input feature
- The two most correlated input features
- All input features

These models will be compared based on their prediction performance for **Heating Load**.

3 Tasks

3.1 Data Splitting

Split the dataset into:

- 60% Training Data : `energyefficient_train.csv`
- 20% Validation Data : `energyefficient_val.csv`
- 20% Test Data : `energyefficient_test.csv`

4 Non-Linear Regression for All 9 Features

Now, instead of using only the top-2 features, repeat each of the three models below using **all 9 input features**.

4.1 Polynomial Regression (All 9 Features)

1. Build a **multivariate polynomial regression model** using all 9 input features with polynomial degrees:

$$p = 1, 2, 3, 4, 5, 6$$

2. Compute **RMSE** on the training and validation datasets for each polynomial degree.
3. Select the polynomial degree with the **lowest validation RMSE** and evaluate the model on the test dataset.
4. Compute **RMSE** on the test dataset.
5. Plot a **line graph**: Training RMSE and Validation RMSE vs Polynomial Degree.
6. Plot a **scatter plot**: Actual vs Predicted heating load on the test dataset for the best polynomial degree.

4.2 Ridge Regression (Linear Model for Regression)

1. Observe the overfitting on the **last polynomial degree** (i.e., $p = 6$) . Now, Construct the corresponding polynomial feature for the last polynomial degree set using all 9 input features.
2. Build a **Ridge Regression model** (linear model with ℓ_2 regularization) on these transformed features.
3. Train the model using this value of the regularization parameter:

$$\alpha = 10^{-2}$$

4. For this value of α , compute the **RMSE** on both the training and validation datasets.
5. Using the α , evaluate the model on the test dataset.
6. Compute the **RMSE** on the test dataset.
7. Plot a **bar graph**: Training RMSE and Validation RMSE vs Polynomial Degree = 6, with and without α .
8. Plot a bar graph to compare with the best polynomial degree previously found.
9. Plot a **scatter plot**: Actual vs Predicted heating load on the test dataset after the use of α .

4.3 Neural Network (All 9 Features)

1. Use **all 9 input features** as the independent variables to predict the heating load.
2. Build a neural network using PyTorch with the following architecture:
 - Input layer: 9 linear neurons (one per feature)
 - Hidden layer: 1 layer with tanh activation function
 - Hidden neurons: {4, 8, 16, 32, 64}
 - Output layer: 1 linear neuron (regression output)
 - Loss function: Mean Squared Error (MSE)
 - Optimizer: Stochastic Gradient Descent (SGD)
 - Learning Rate: 0.01
 - Epochs: 700
3. Train the neural network with varying hidden neurons and compare the RMSE on training and validation datasets for all the architectures.
4. Plot **Training and Validation Loss vs Epochs** for all the neural network architectures.

5. Select the network configuration with the **lowest validation RMSE** and evaluate it on the test dataset.
6. Compute **RMSE** on the test dataset.
7. Plot a **scatter plot**: Actual vs Predicted heating load on the test dataset for the best neural network.

5 Non-Linear Regression for Top-2 Features

Using the **two attributes that have the highest correlation** with the target variable (Heating Load) on the training set, perform the following tasks.

5.1 Polynomial Regression (Top 2 Features)

1. Build a **multivariate polynomial regression model** using the two selected attributes with polynomial degrees:

$$p = 1, 2, 3, 4, 5, 6$$

2. Compute **RMSE** on the training and validation datasets.
3. Plot the **best-fit surface** using the training data and predicted data for each polynomial degree.
 - x-axis: Feature 1
 - y-axis: Feature 2
 - z-axis: Heating Load
4. Select the polynomial degree with the **lowest validation RMSE** and evaluate the model on the test dataset.
5. Compute **RMSE** for the test dataset.
6. Plot a **line graph**: Training RMSE and Validation RMSE vs Polynomial Degree.
7. Plot a **scatter plot**: Actual vs Predicted heating load on the test dataset for the best polynomial degree.

5.2 Neural Network (Top 2 Features)

1. Use the **top-2 input features** as the independent variables to predict the heating load.
2. Build a neural network using PyTorch with the following architecture:

- Input layer: 2 linear neurons (one per feature)
 - Hidden layer: 1 layer with tanh activation function
 - Hidden neurons: {4, 8, 16, 32, 64}
 - Output layer: 1 linear neuron (regression output)
 - Loss function: Mean Squared Error (MSE)
 - Optimizer: Stochastic Gradient Descent (SGD)
 - Learning Rate: 0.01
 - Epochs: 700
3. Train the neural network with varying hidden neurons and compare the RMSE on training and validation datasets for all the architectures.
 4. Plot **Training and Validation Loss vs Epochs** for all the neural network architectures.
 5. Plot the **best-fit surface** using training and predicted data for each neural network architecture.
 - x-axis: Feature 1
 - y-axis: Feature 2
 - z-axis: Heating Load
 6. Select the architecture with the **lowest validation RMSE** and evaluate it on the test dataset.
 7. Compute **RMSE** on the test dataset.
 8. Plot a **scatter plot**: Actual vs Predicted energy load on the test dataset.

Sample Code Snippets for Regression Tasks

The following code blocks demonstrate how to implement the required correlation analysis, simple linear regression, and polynomial curve fitting using Python and scikit-learn.

1. **Correlation Analysis and Feature Selection:** Compute correlations with the target variable, identify the best feature, and prepare the splits.

```

1 import pandas as pd
2
3 # 1. Compute correlations with the target variable 'CO'
4 correlations = df.corr()['CO'].drop('CO')
5
6 # 2. Find the feature with the highest absolute correlation (
7     x_best)
8 x_best = correlations.abs().idxmax()

```

```

8 print(f"Feature with highest absolute correlation: {x_best}")
9
10 # 3. Define independent variables using x_best
11
12 X_train = train_df[[x_best]]
13 y_train = train_df['CO']
14
15 X_val = val_df[[x_best]]
16 y_val = val_df['CO']
17
18 X_test = test_df[[x_best]]
19 y_test = test_df['CO']

```

2. Ridge Regression (Degree = 4):

```

1 from sklearn.linear_model import Ridge
2 from sklearn.preprocessing import PolynomialFeatures
3 from sklearn.metrics import mean_squared_error
4 import numpy as np
5
6 # Polynomial transformation
7 poly = PolynomialFeatures(degree=4)
8 X_train_poly = poly.fit_transform(X_train)
9 X_val_poly = poly.transform(X_val)
10 X_test_poly = poly.transform(X_test)
11
12 # Ridge Regression model
13 ridge = Ridge(alpha=1)
14 ridge.fit(X_train_poly, y_train)
15
16 # Predictions
17 y_train_pred = ridge.predict(X_train_poly)
18 y_val_pred = ridge.predict(X_val_poly)
19 y_test_pred = ridge.predict(X_test_poly)
20
21 # RMSE calculation
22 rmse_train = np.sqrt(mean_squared_error(y_train, y_train_pred))
23 rmse_val = np.sqrt(mean_squared_error(y_val, y_val_pred))
24 rmse_test = np.sqrt(mean_squared_error(y_test, y_test_pred))
25
26 print(f"Train RMSE: {rmse_train:.4f} | Val RMSE: {rmse_val:.4f}
      | Test RMSE: {rmse_test:.4f}")

```

3. Neural Network Regression (PyTorch):

```
1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4
5 class Net(nn.Module):
6     def __init__(self, hidden_size):
7         super(Net, self).__init__()
8         self.fc1 = nn.Linear(2, hidden_size)
9         self.tanh = nn.Tanh()
10        self.fc2 = nn.Linear(hidden_size, 1)
11
12    def forward(self, x):
13        x = self.tanh(self.fc1(x))
14        x = self.fc2(x)
15        return x
16
17 hidden_sizes = [16, 32, 64]
18
19 for h in hidden_sizes:
20     model = Net(h)
21     criterion = nn.MSELoss()
22     optimizer = optim.SGD(model.parameters(), lr=0.001)
23
24     for epoch in range(500):
25         optimizer.zero_grad()
26         outputs = model(X_train_tensor)
27         loss = criterion(outputs, y_train_tensor)
28         loss.backward()
29         optimizer.step()
```

4. 3D Best-Fit Surface (Linear Regression with Top 2 Features):

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.linear_model import LinearRegression
4
5 # Train Linear Regression model
6 model = LinearRegression()
7 model.fit(X_train_top2, y_train)
8
9 # Feature names
10 f1, f2 = X_train_top2.columns
11
12 # Create grid over feature space
13 x1 = np.linspace(X_train_top2[f1].min(), X_train_top2[f1].max(),
14                  , 100)
```

```

14 x2 = np.linspace(X_train_top2[f2].min(), X_train_top2[f2].max()
    , 100)
15
16 X1_grid, X2_grid = np.meshgrid(x1, x2)
17
18 # Predict surface
19 grid_points = np.c_[X1_grid.ravel(), X2_grid.ravel()]
20 Z_pred = model.predict(grid_points)
21 Z_pred = Z_pred.reshape(X1_grid.shape)
22
23 # 3D Plot
24 fig = plt.figure(figsize=(10,7))
25 ax = fig.add_subplot(111, projection='3d')
26
27 # Plot surface
28 surface = ax.plot_surface(X1_grid, X2_grid, Z_pred,
29                           cmap='viridis', alpha=0.7)
30
31 # Plot training data
32 ax.scatter(X_train_top2[f1],
33            X_train_top2[f2],
34            y_train,
35            color='red', s=20, label='Training Data')
36
37 # Labels and title
38 ax.set_xlabel(f1)
39 ax.set_ylabel(f2)
40 ax.set_zlabel("Heating Load")
41 ax.set_title("3D Best-Fit Surface (Linear Regression)")
42
43 # Colorbar and view
44 fig.colorbar(surface, ax=ax, shrink=0.5, aspect=10)
45 ax.view_init(elev=30, azim=45)
46
47 ax.legend()
48 plt.show()
49
50 # Print model equation
51 w1, w2 = model.coef_
52 b = model.intercept_
53
54 print(f"z = {w1:.4f}*{f1} + {w2:.4f}*{f2} + {b:.4f}")

```

5. Bar Plot Comparison (Degree = 4: Linear vs Ridge):

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3

```



```

4 # Extract degree 4 rows
5 deg6_linear = results_df[(results_df["Degree"] == 4) & (
    results_df["Model"] == "Linear")]
6 deg6_ridge = results_df[(results_df["Degree"] == 4) & (
    results_df["Model"] == "Ridge(1e-2)")]
7
8 # Labels
9 labels = ["Linear (No Reg)", "Ridge (alpha=1e-2)"]
10
11 # Values
12 train_vals = [
13     deg6_linear["Train RMSE"].values[0],
14     deg6_ridge["Train RMSE"].values[0]
15 ]
16
17 val_vals = [
18     deg6_linear["Validation RMSE"].values[0],
19     deg6_ridge["Validation RMSE"].values[0]
20 ]
21
22 # X positions
23 x = np.arange(len(labels))
24 width = 0.35
25
26 # Plot
27 plt.figure(figsize=(7,5))
28
29 plt.bar(x - width/2, train_vals, width, label="Train RMSE")
30 plt.bar(x + width/2, val_vals, width, label="Validation RMSE")
31
32 plt.xticks(x, labels)
33 plt.ylabel("RMSE")
34 plt.title("Degree 4: Linear vs Ridge Comparison")
35
36 plt.legend()
37 plt.grid(True, axis='y')
38 plt.tight_layout()
39
40 plt.show()

```